PySide6 + SQLite

SQLite viene integrado en Qt:

```
from PySide6.QtSql import QSqlDatabase
print(QSqlDatabase.drivers()) // comprueba que existe el driver QSQLITE

db = QSqlDatabase.addDatabase("QSQLITE")
db.setDatabaseName("torneoFutbol_sqlite.db") // Si no existe el fichero .db lo crea

if not db.open():
    print("Error al abrir la BBDD")
else:
    print("BBDD abierta correctamente")
```

SQLite no activa las foreing Keys por defecto, después de abrir la base de datos:

```
from PySide6.QtSql import QSqlQuery
query = QSqlQuery()
query.exec("PRAGMA foreign_keys = ON;")
```

Generación de tablas:

```
query.exec("""
        CREATE TABLE IF NOT EXISTS participantes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nombre TEXT NOT NULL,
            fecha_nacimiento TEXT NOT NULL,
            curso TEXT NOT NULL,
            tipo_jugador TEXT NOT NULL,
            posicion TEXT NOT NULL,
            t_amarillas INTEGER DEFAULT 0,
            t_rojas INTEGER DEFAULT 0,
            goles INTEGER DEFAULT 0
```

```
        ) """)
```

Se debe generar un fichero **database.py** dentro de la carpeta models con todo lo mencionado anteriormente. Recordad que dentro del fichero main.py solamente se arranca la aplicación.

Ejemplo de database.py:

```
from PySide6.QtSql import QSqlDatabase, QSqlQuery


def conectar():
    # Crear la conexión a la base de datos
    db = QSqlDatabase.addDatabase("QSQLITE")
    db.setDatabaseName("torneoFutbol_sqlite.db")

    if not db.open():
        raise Exception("No se pudo abrir la BD")

    # Crear un objeto query
    query = QSqlQuery()
    query.exec("PRAGMA foreign_keys = ON;")

    crear_tablas(query)

    return db


def crear_tablas(query):
    query.exec("""
    CREATE TABLE IF NOT EXISTS jugador (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        fecha_nacimiento TEXT NOT NULL,
```

```
        curso TEXT NOT NULL,
        tipo_jugador TEXT NOT NULL,
        posicion TEXT NOT NULL,
        t_amarillas INTEGER DEFAULT 0,
        t_rojas INTEGER DEFAULT 0,
        goles INTEGER DEFAULT 0
    )
    """)
```

Cargar QSS

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from views.mainwindow import Ui_MainWindow

def load_stylesheet(path):
    # Carga el archivo QSS y lo devuelve como string
    with open(path, "r") as file:
        return file.read()

if __name__ == "__main__":
    app = QApplication(sys.argv)

    # Cargar QSS
    qss = load_stylesheet("resources/style.qss")
    app.setStyleSheet(qss)
```

Distribución de la aplicación

Integración del fichero . db en el ejecutable:

pyinstaller --add-data **origen;destino**

pyinstaller --add-data "tarea3_torneoFutbol/models/torneoFutbol_sqlite.db;tarea3/models" tarea3_torneoFutbol/main.py

Se puede añadir:

--onefile para hacer que esté todo en un fichero

--windowed para eliminar la consola